Recursion Lab

This lab has five recursive functions to write, plus one bonus problem. Your TA will work through the first two with you; then you continue in pairs.

How the lab is organized

Each problem lives in its own folder, `ch1` through `ch6`. Every folder contains a `Main.java` with:

- a `main` method that asks you for an input and runs your solution on it, and
- a method stub marked `//TODO` that you need to complete.

Compile and run a problem from inside its folder:

```
javac Main.java
java Main
```

The scaffold compiles and runs before you have written anything, because the stub simply returns a placeholder value.

Every folder except `ch6` also contains a `Test.java`. It runs your solution on a whole list of inputs at once and reports whether each one gave the answer you expected. Run it from inside the folder:

```
javac Test.java
java Test
```

You do not need to compile `Main.java` separately; `javac` finds it for you.

Each problem below ends with a short list of cases worth thinking about. You can add them to `Test.java` as you go.

ch1 — `collatzLength`

The Collatz conjecture is a deceptively simple problem in number theory. Consider the following recurrence relation, applied repeatedly starting from any positive integer $a_0 = n$:

$$a_{i+1} = \begin{cases} a_i/2 & \text{if } a_i \text{ is even} \\ 3a_i + 1 & \text{if } a_i \text{ is odd} \end{cases}$$

Which means: if `n` is even, divide it by 2; if it is odd, multiply it by 3 and add 1. The conjecture asserts that no matter what `n` you start from, by repeatedly applying these operations you always eventually reach 1. While this has never been proven, it has also never failed for any number anyone has tried!

Write `collatzLength(n)`, which returns how many steps it takes to reach 1 from `n`. `collatzLength(1)` is 0, since you are already there. If `n` is zero or negative, throw an `IllegalArgumentException` instead of trying to compute anything.

- `collatzLength(5)` returns 5: `5, 16, 8, 4, 2, 1`.
- `collatzLength(6)` returns 8: `6, 3, 10, 5, 16, 8, 4, 2, 1`.

Testing. Does your function work if you give it:

- `n = 1`?
- a case with an odd step and a case with an even step, small enough to trace by hand? (You do not need to trace a large `n` by hand. It might be hard to predict how many steps a number will take.)
- a zero or negative number?

ch2 — `fib`

The Fibonacci numbers are the sequence starting $0, 1, 1, 2, 3, 5, 8, 13, \ldots$, where each number is the sum of the two before it: $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Write a method `fib(n)` that computes F_n. If `n` is negative, throw an `IllegalArgumentException` instead of trying to compute anything.

`Main.java` also contains `fibIterative(n)`, already complete, and prints how long each version takes. Try a larger `n` and watch what happens. *Why is the recursive version slower? How would you make it faster but still recursive?* (You don't have to do it now, we will discuss a solution in class.)

ch3 — `binom`

The binomial coefficient $\binom{n}{k}$ counts how many ways there are to choose k items out of n. Pascal's rule gives a recursive definition: $\binom{n}{0} = \binom{n}{n} = 1$, and otherwise $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.

Write `binom(n, k)` directly from that rule, for `0 <= k <= n`. If `n` or `k` is negative, or `k > n`, throw an `IllegalArgumentException` instead of trying to compute anything.

Testing. Does your function work if you give it:

- `k = 0` or `k = n`?
- `n` and `k` both 0?
- a case in the middle, such as `binom(6, 3)`?
- What do you notice if you try a `binom(n, k)` with `n` around 35 or 40?
- a negative `n` or `k`, or a `k` greater than `n`?

ch4 — `isPalindromeRecursive`

A palindrome reads the same forwards and backwards, ignoring case, spaces, and punctuation: only letters and digits count. `"Racecar"` and `"A man, a plan, a canal: Panama"` are both palindromes.

`Main.java` contains `clean(s)`, already complete, which strips everything but letters and digits and lowercases what is left. Read it to see how it works. Write `isPalindromeRecursive(s)` in terms of it: clean the string once, then recurse on the result.

`Main.java` also contains `isPalindromeIterative(s)`, already complete, which solves the same problem with a loop instead. Once your tests pass, compare the two. *What is different between them, in what they do, and in what they cost?*

Hint. A cleaned string is a palindrome if it has 0 or 1 characters, or if its first and last characters match *and* the substring strictly between them is a palindrome. Recursing on that substring costs an extra copy on every call, which is not the most efficient approach, but it is the cleanest one and it's OK for the submission. *Think of how you could make it more efficient.*

Testing. Does your function work if you give it:

- the empty string?
- a single character?
- a string that is not a palindrome at all?
- a string that would be a palindrome only once you drop punctuation and spacing?
- `null`? (this should throw an `IllegalArgumentException`, you get this for free by going through `clean`)

ch5 — `ruler`

A ruler of height `h` is built like this: draw a ruler of height `h-1`, then a tick of height `h`, then another ruler of height `h-1`. A ruler of height 0 is empty.

Write `ruler(h)`, which returns the whole ruler as a single `String`, one tick per line. A tick of height `k` is `k` dashes followed by a newline. If `h` is negative, throw an `IllegalArgumentException` instead of trying to compute anything.

Hint. `"-".repeat(k)` gives you a tick of height `k`; add `"\n"` to end its line.

You can build the string with `String` concatenation, which, as with the substring approach in ch4, is not the fastest way. To make it faster, you can use the `StringBuilder` class; we will cover it properly once we get to array-based lists.

- `ruler(2)` returns the string `"-\n--\n-\n"`, which prints as:

  ```
  -
  --
  ```

-

Testing. Does your function work if you give it:

- `h = 0`? (this should return the empty string, not a string containing a newline)
- `h = 1`?
- a height a few steps larger than the ones already in `Test.java`?
- a negative height?

ch6 — `binarySequences` (bonus, not required)

Write `binarySequences(n)`, which returns every binary string of length `n` as a `String` array. For instance, `binarySequences(2)` should return an array holding `"00"`, `"01"`, `"10"`, and `"11"`, in some order. `Main.java` already prints the array it gets back. If `n` is negative, throw an `IllegalArgumentException` instead of trying to compute anything.

Note. `binarySequences(0)` should return an array holding a single empty string, not an empty array, but `Main.java` prints both of those identically as `[]`, since printing an empty string prints nothing. Check `.length` if you want to tell them apart.